МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Кафедра защиты информации



ЛАБОРАТОРНАЯ РАБОТА №2

по дисциплине: «*Безопасность систем баз данных*»

Вариант: Защита БД образовательных учреждений

Выполнил:

Студент гр. «АБ-420», «АВТФ»

Чубреич Я. С.

«__» _____ 2026г

_____

(подпись)

Проверил:

Старший преподаватель ЗИ

Питько Я. А.

«___» _____ 2026г

_____

(подпись)

Новосибирск 2026

SECURITY DEFINER-функции (точечное повышение)

Цель: Реализовать безопасные операции через функции с повышенными привилегиями, сравнить методы контроля бизнес-правил.

Были реализованы 3 функции для чувствительных операций в предметной области «Защита БД образовательных учреждений». Код всех 3 функций приведен в Приложении А.

Первая функция (app.update_person_contacts()) позволяет роли app_writer изменять телефон и email в app.personal_files, не выдавая прямой доступ к таблице. Функция проверяет корректность идентификатора, телефона и email.

Вторая функция (app.delete_grade()) используется для контролируемого удаления оценок. Прямой DELETE для app_writer запрещён, а функция дополнительно не позволяет удалять итоговые оценки типа final.

Третья функция (app.enroll_student()) выполняет безопасную запись студента на проведение дисциплины. Проверяются существование и статус студента, соответствие учебной группе и отсутствие повторной записи.

Тесты прав

Проведена проверка, что роль app_reader имеет доступ к чтению разрешённых данных (Рисунок 1).

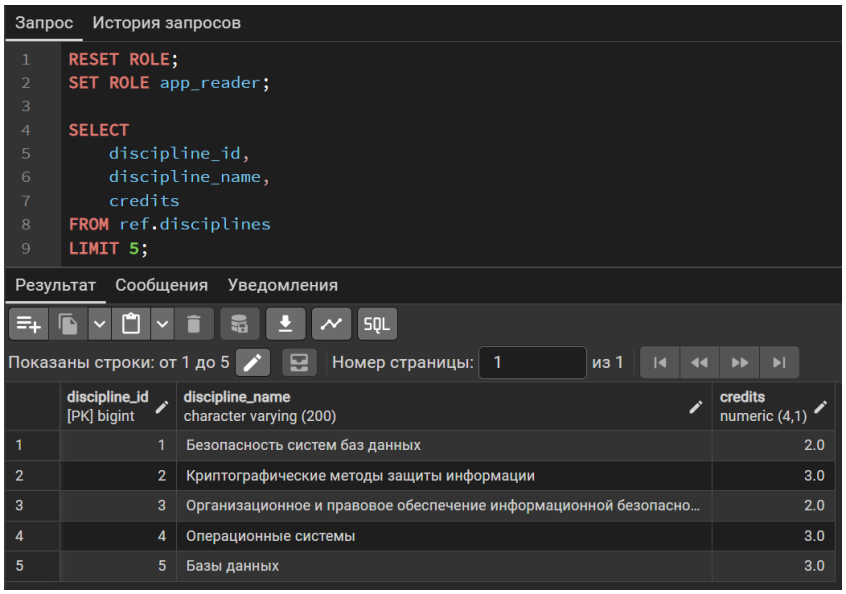


Рисунок 1 – Проверка доступа роли app_reader к чтению данных

Проведена проверка, что роль app_reader не может добавлять данные в таблицы (Рисунок 2).

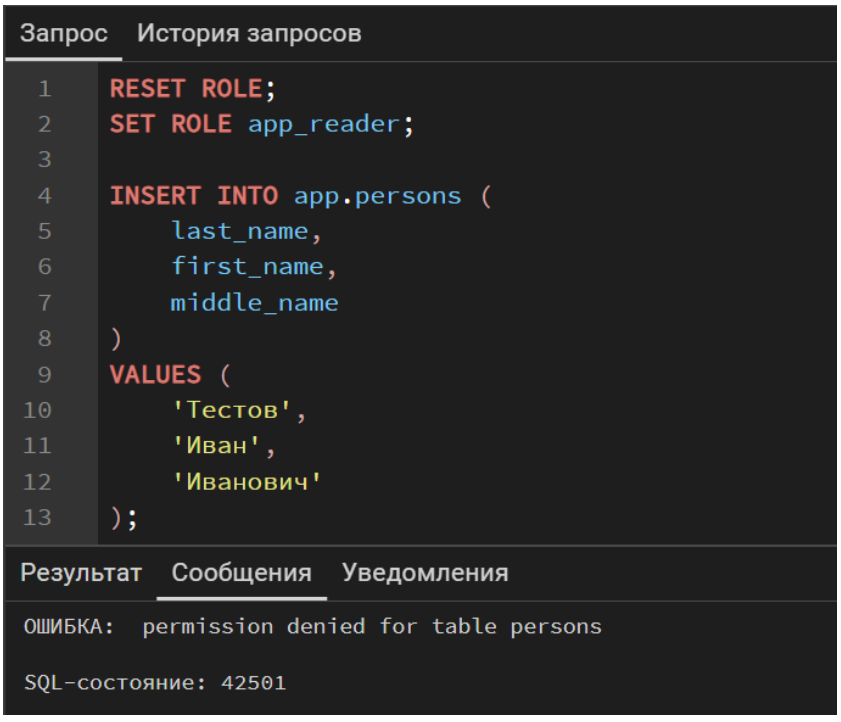


Рисунок 2 – Проверка запрета добавления данных для роли app_reader

Проведена проверка, что роль auditor имеет доступ к чтению журнала вызова защищённых функций (Рисунок 3).

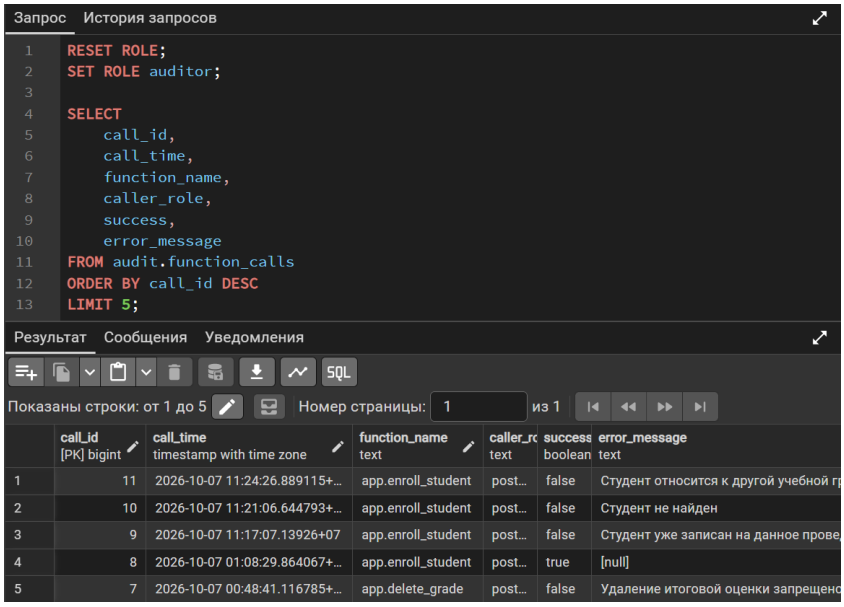


Рисунок 3 – Проверка доступа роли auditor к журналу вызова функций

Проведена проверка, что роль auditor не может изменять данные журнала аудита (Рисунок 4).

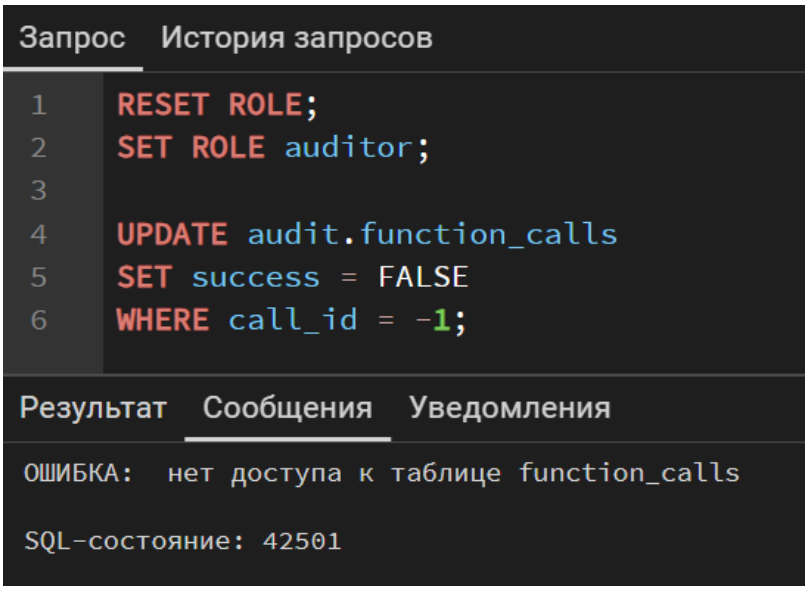


Рисунок 4 – Проверка запрета изменения журнала аудита для роли auditor

Проведена проверка, что роль app_writer не может напрямую изменять чувствительные данные в таблице app.personal_files (Рисунок 5).

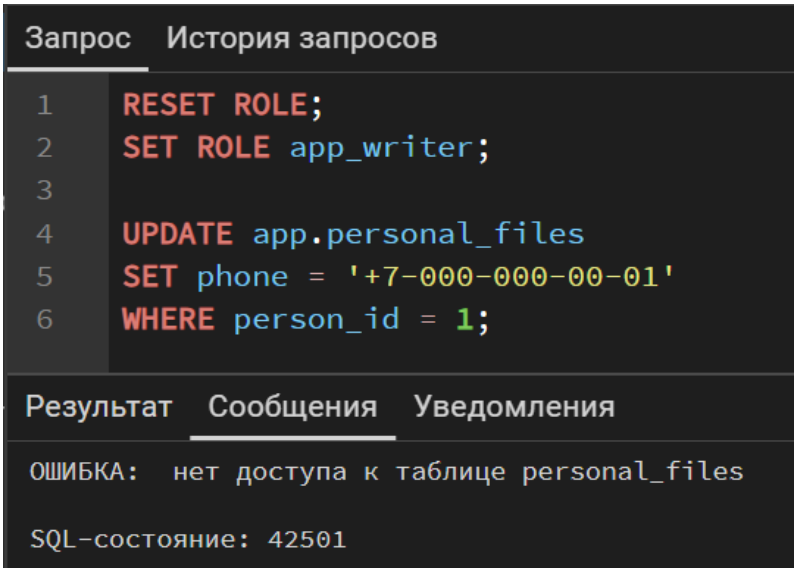


Рисунок 5 – Проверка запрета изменения персональных данных для роли app_writer

Проведена проверка, что роль app_writer может изменить контактные данные через защищённую функцию без прямого доступа к таблице app.personal_files (Рисунок 6).

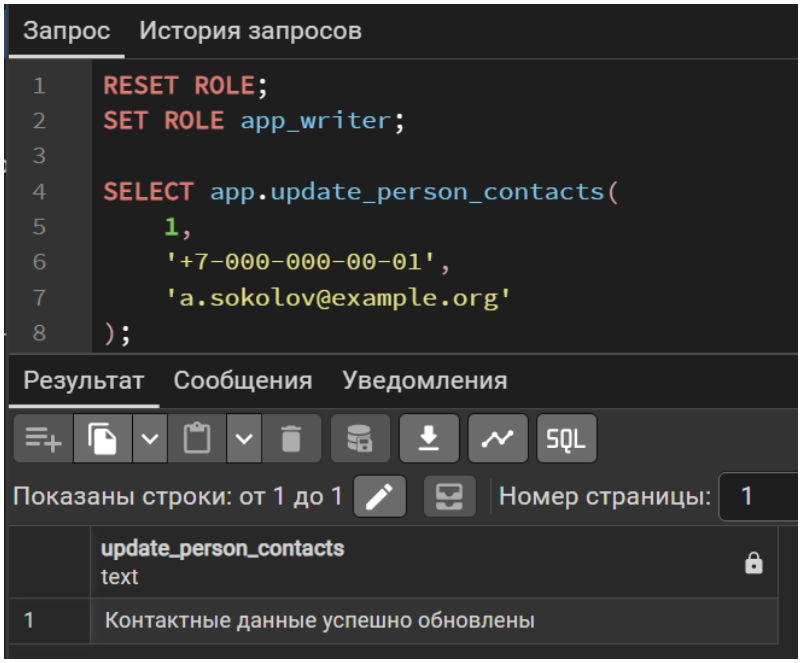


Рисунок 6 – Проверка изменения контактных данных ролью app_writer через защищенную функцию

Проведена проверка обработки невалидных данных функцией app.update_person_contacts() (Рисунок 7).

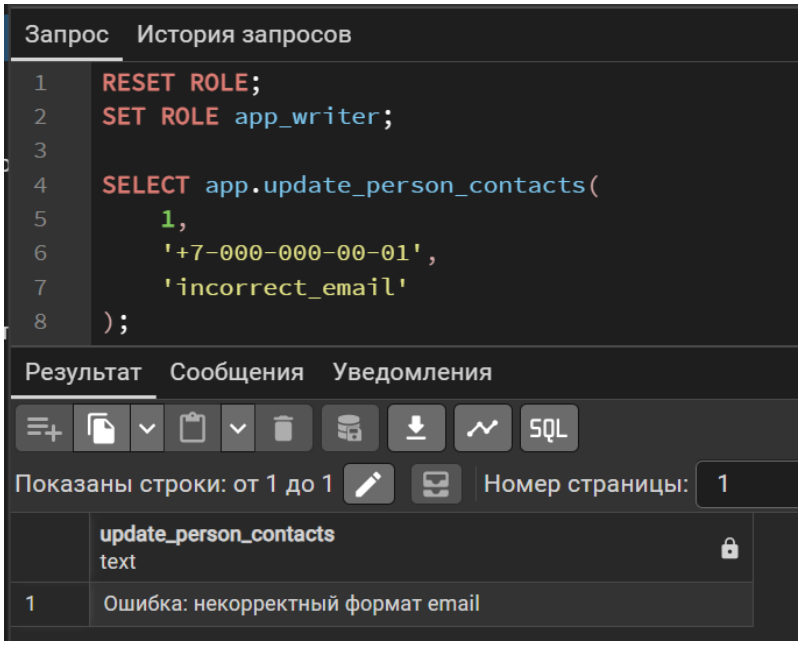


Рисунок 7 – Проверка обработки невалидных данных функцией app.update_person_contacts()

Проведена проверка, что роль app_writer не может напрямую удалять записи из таблицы app.grades (Рисунок 8).

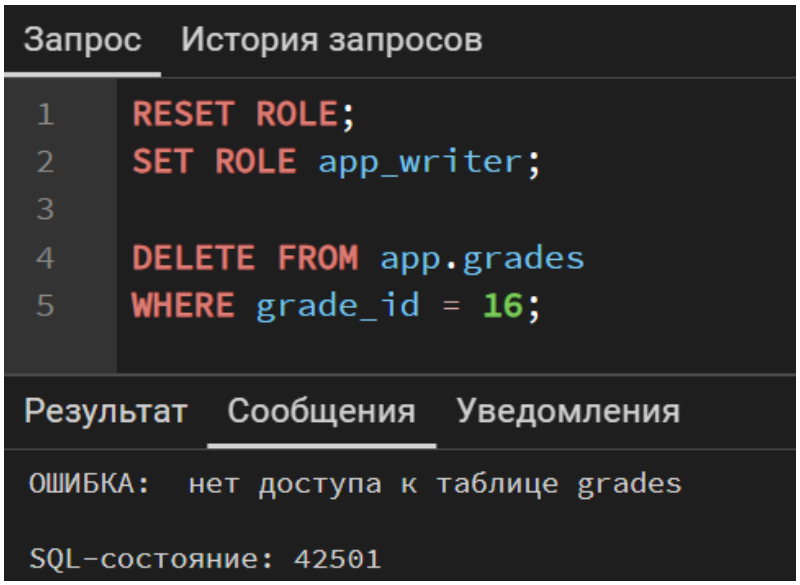


Рисунок 8 – Проверка запрета прямого удаления оценок для роли app_writer

Проведена проверка, что роль app_writer может удалить обычную оценку только через защищённую функцию (Рисунок 9).

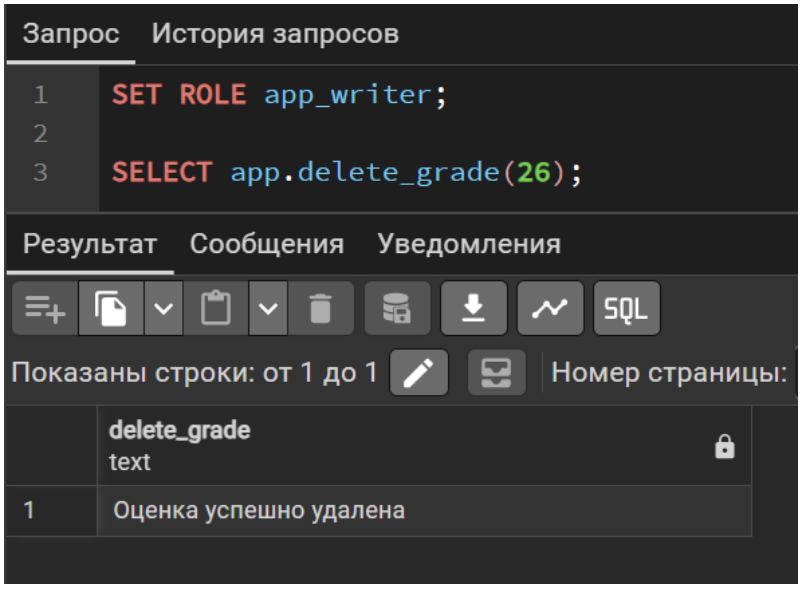


Рисунок 9 – Проверка удаления оценки через защищенную функцию

Проведена проверка, что функция app.delete_grade() запрещает удаление итоговой оценки (Рисунок 10).

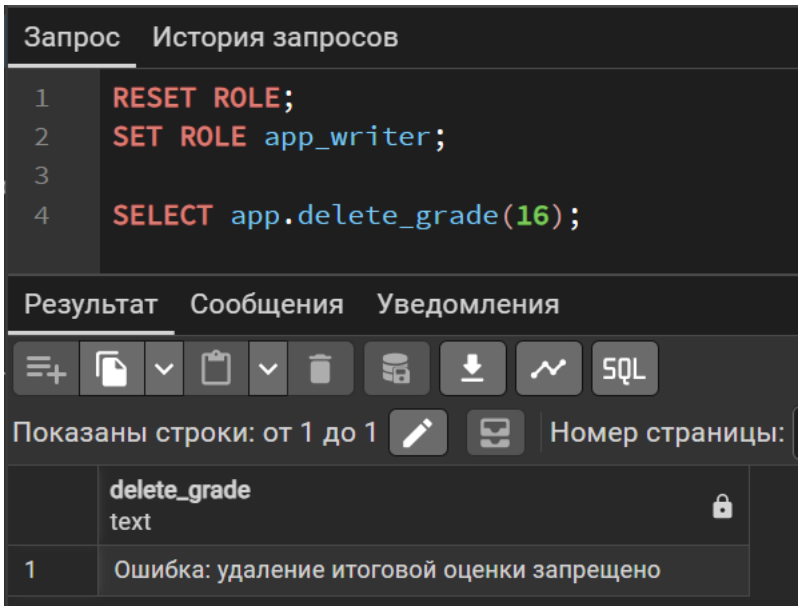


Рисунок 10 – Проверка запрета удаления итоговой оценки защищенной функцией

Проведена проверка, что роль app_writer не имеет права напрямую добавлять записи в таблицу app.enrollments (Рисунок 11).

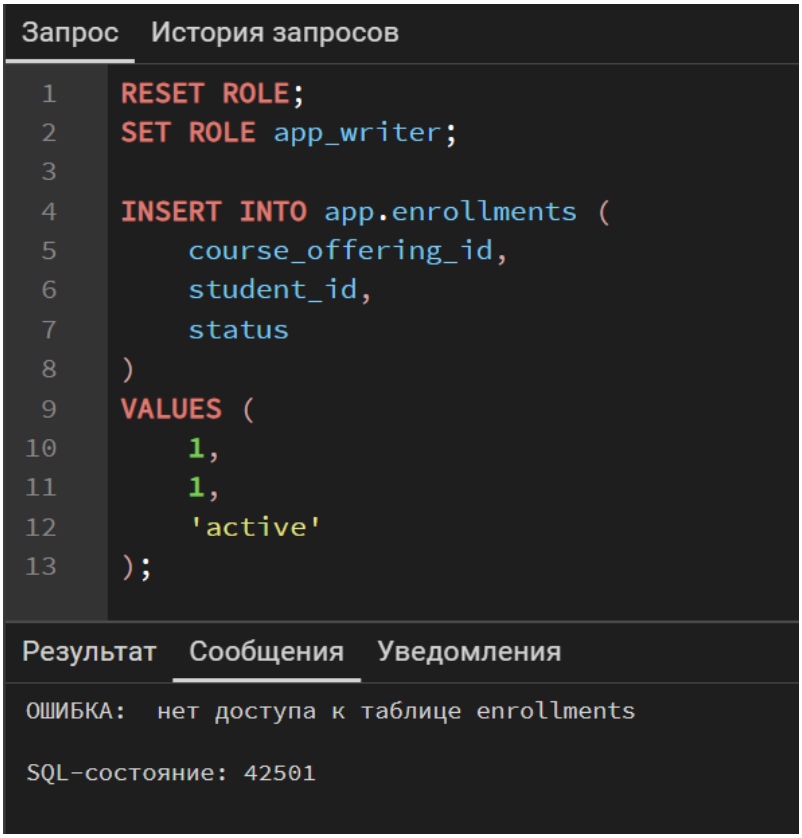


Рисунок 11 – Проверка запрета прямого добавления записей в app.enrollments для роли app_writer

Проведена проверка, что роль app_writer может записать студента на проведение дисциплины через защищенную функцию (Рисунок 12).

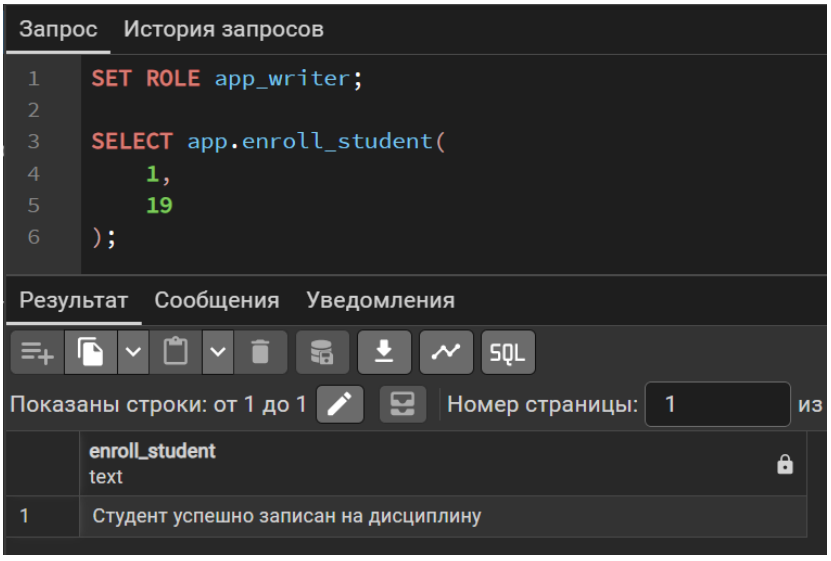


Рисунок 12 – Проверка записи студента ролью app_writer через функцию

Проведена проверка, что защищенная функция не допускает повторную запись одного студента на одно и то же проведение дисциплины (Рисунок 13).

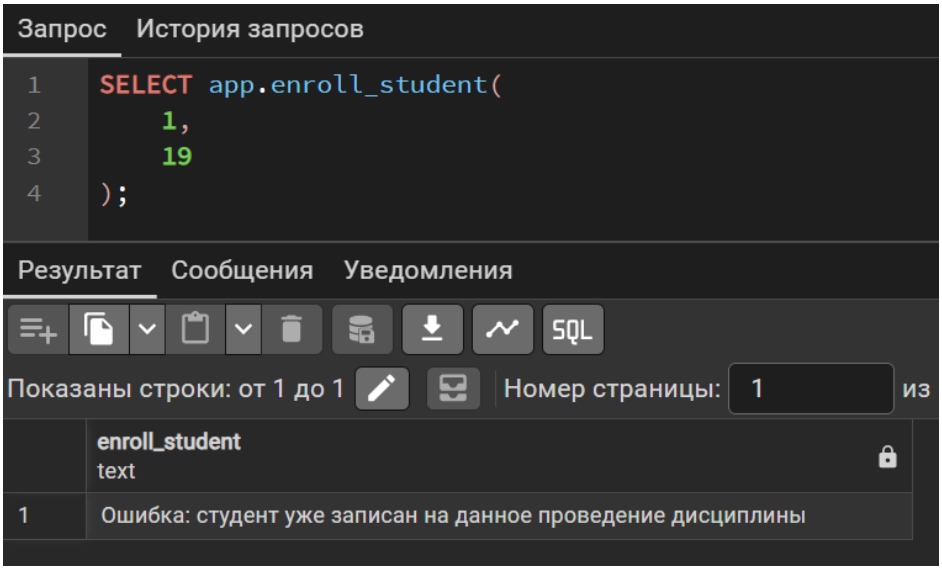


Рисунок 13 – Проверка запрета повторной записи студента ролью app_writer на дисциплину

Проведена проверка, что функция app.enroll_student() запрещает запись студента на проведение дисциплины другой учебной группы (Рисунок 14).

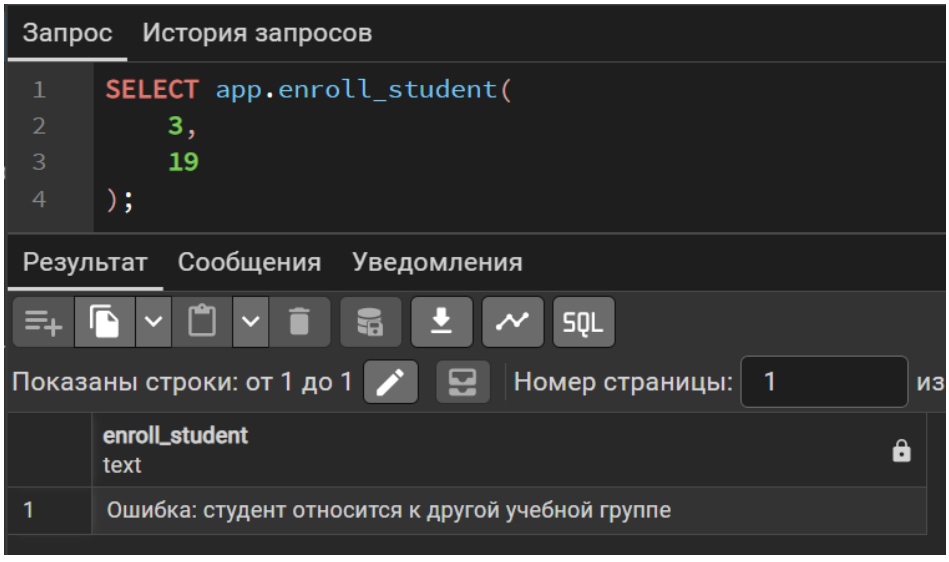


Рисунок 14 – Проверка контроля соответствия учебной группы защищенной функцией

Проведена проверка, что роль app_reader не может выполнять DDL-операции (Рисунок 15).

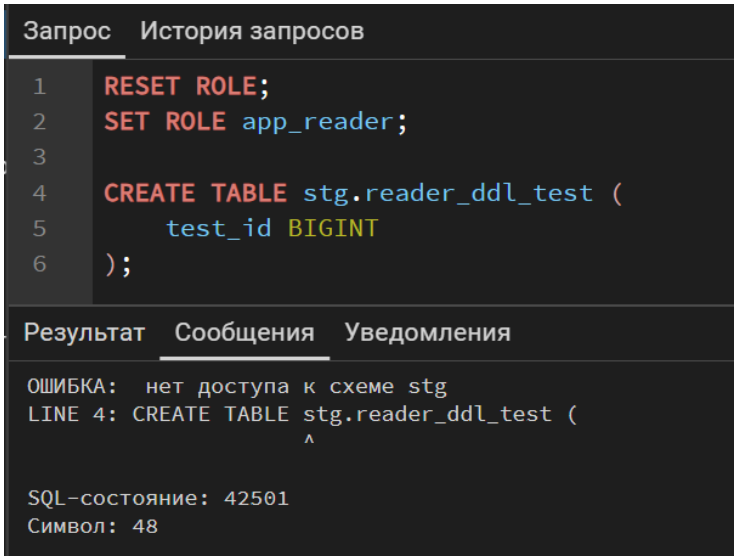


Рисунок 15 – Проверка запрета выполнения DDL-операций для роли app_reader

Проведена проверка, что роль app_writer не имеет права напрямую добавлять записи в журнал audit.function_calls (Рисунок 16).

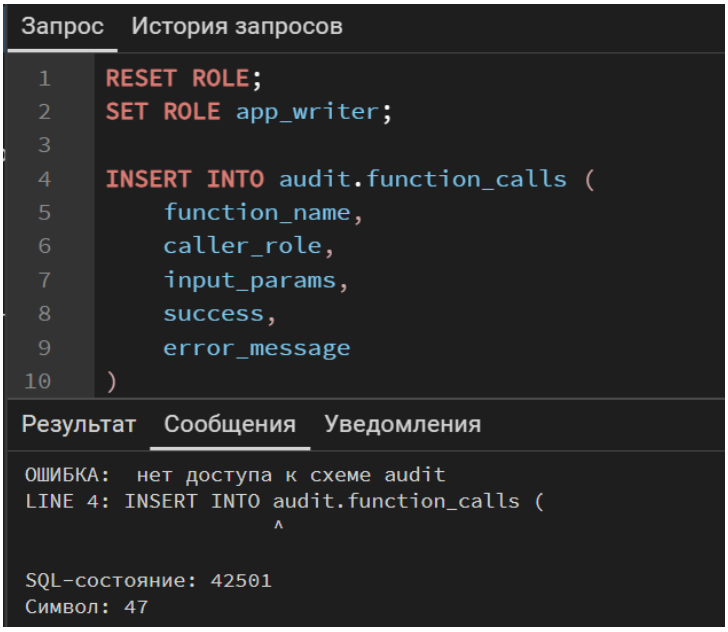


Рисунок 16 – Проверка запрета прямой записи в журнал вызовов функций для роли app_writer

Проверки показали, что все права реализованы корректно.

CHECK vs триггер

Для сравнения CHECK и триггера было выбрано следующие бизнес-правило: номер семестра должен быть 1 или 2, дата окончания должна быть позже даты начала, а продолжительность семестра – не более 200 дней. Код создания тестовых таблиц, функции проверки бизнес-правила, триггера приведен в Приложении В.

Была проведена проверка корректности CHECK и триггера путем вставки валидных значений (Рисунок 17–18).

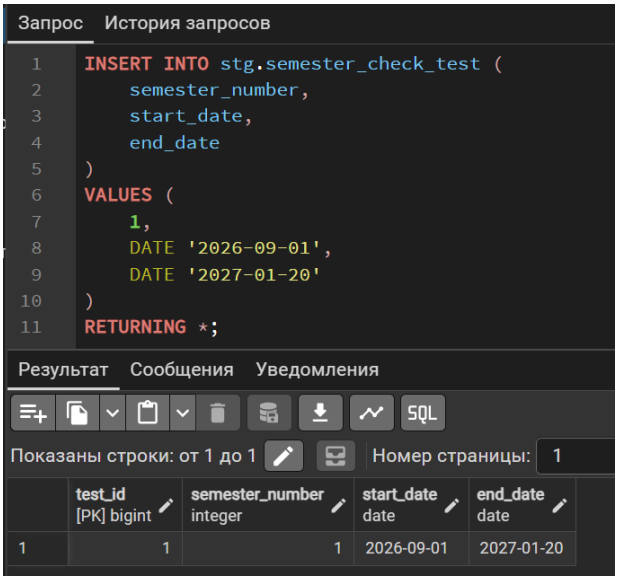


Рисунок 17 – Проверка корректности CHECK вставкой валидных значений

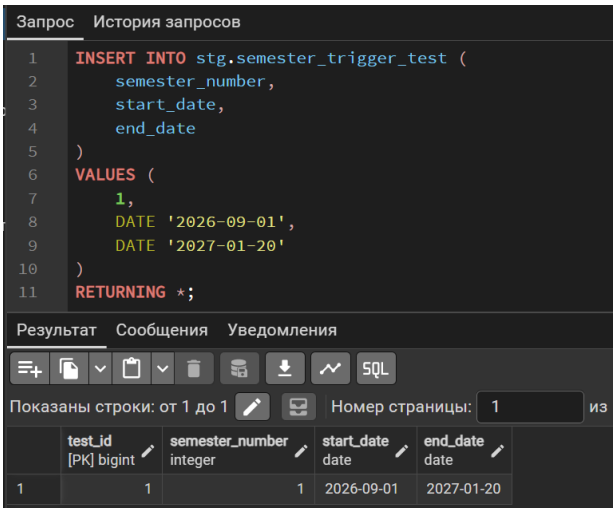


Рисунок 18 – Проверка корректности триггера вставкой валидных значений

Была проведена проверка корректности CHECK и триггера путем вставки невалидного номера семестра (Рисунок 19–20).

Запрос История запросов

```
1  INSERT INTO stg.semester_check_test (
2      semester_number,
3      start_date,
4      end_date
5  )
6  VALUES (
7      3,
8      DATE '2026-09-01',
9      DATE '2027-01-20'
10 );
```

Результат Сообщения Уведомления

```
ОШИБКА:  новая строка в отношении "semester_check_test" нарушает ограничение-проверку
"chk_semester_rules"
Ошибочная строка содержит (2, 3, 2026-09-01, 2027-01-20).

SQL-состояние: 23514
Подробности: Ошибочная строка содержит (2, 3, 2026-09-01, 2027-01-20).
```

Рисунок 19 – Проверка корректности CHECK вставкой невалидного номера семестра

Запрос История запросов

```
1  INSERT INTO stg.semester_trigger_test (
2      semester_number,
3      start_date,
4      end_date
5  )
6  VALUES (
7      3,
8      DATE '2026-09-01',
9      DATE '2027-01-20'
10 );
```

Результат Сообщения Уведомления

```
ОШИБКА:  Номер семестра должен быть 1 или 2
CONTEXT:  функция PL/pgSQL stg.validate_semester_trigger(), строка 4, оператор RAISE

SQL-состояние: 23514
```

Рисунок 20 – Проверка корректности триггера вставкой невалидного номера семестра

Сравнение проверок показало, что триггер выдает более понятное сообщение об ошибке.

Была проведена проверка корректности CHECK и триггера путем вставки невалидной даты (дата начала больше даты окончания) (Рисунок 21–22).

Запрос | История запросов

```
1  INSERT INTO stg.semester_check_test (
2      semester_number,
3      start_date,
4      end_date
5  )
6  VALUES (
7      1,
8      DATE '2026-09-01',
9      DATE '2026-08-20'
10 );
```

Результат | Сообщения | Уведомления

```
ОШИБКА:  новая строка в отношении "semester_check_test" нарушает ограничение-проверку
"chk_semester_rules"
Ошибочная строка содержит (3, 1, 2026-09-01, 2026-08-20).

SQL-состояние: 23514
Подробности: Ошибочная строка содержит (3, 1, 2026-09-01, 2026-08-20).
```

Рисунок 21 – Проверка корректности CHECK вставкой невалидной даты

Запрос | История запросов

```
1  INSERT INTO stg.semester_trigger_test (
2      semester_number,
3      start_date,
4      end_date
5  )
6  VALUES (
7      1,
8      DATE '2026-09-01',
9      DATE '2026-08-20'
10 );
```

Результат | Сообщения | Уведомления

```
ОШИБКА:  Дата окончания должна быть позже даты начала
CONTEXT:  функция PL/pgSQL stg.validate_semester_trigger(), строка 9, оператор RAISE

SQL-состояние: 23514
```

Рисунок 22 – Проверка корректности триггера вставкой невалидной даты

Сравнение проверок показало, что триггер снова выдает более понятное сообщение об ошибке.

Была проведена проверка корректности CHECK и триггера путем вставки невалидной продолжительности семестра (Рисунок 23–24).

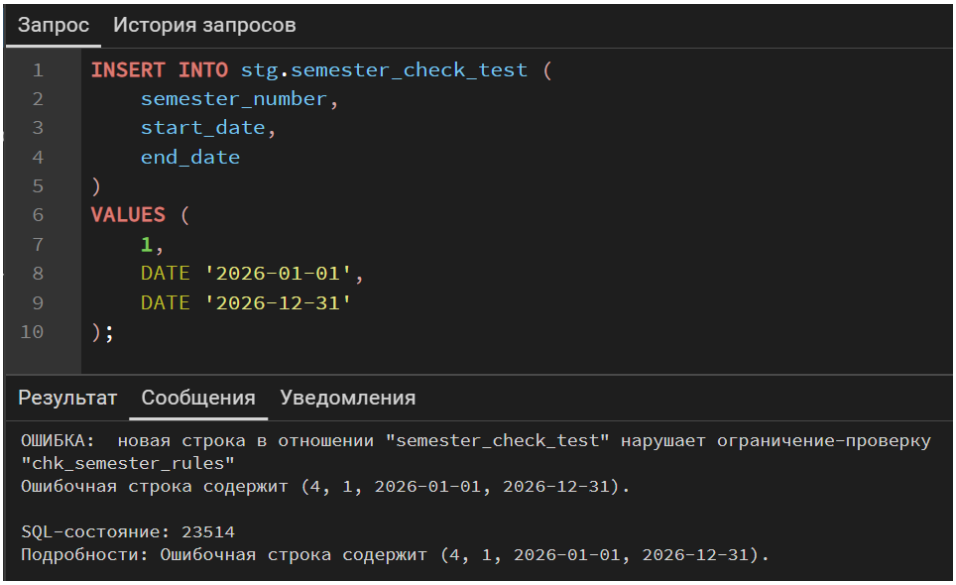


Рисунок 23 – Проверка корректности CHECK вставкой невалидной продолжительности семестра

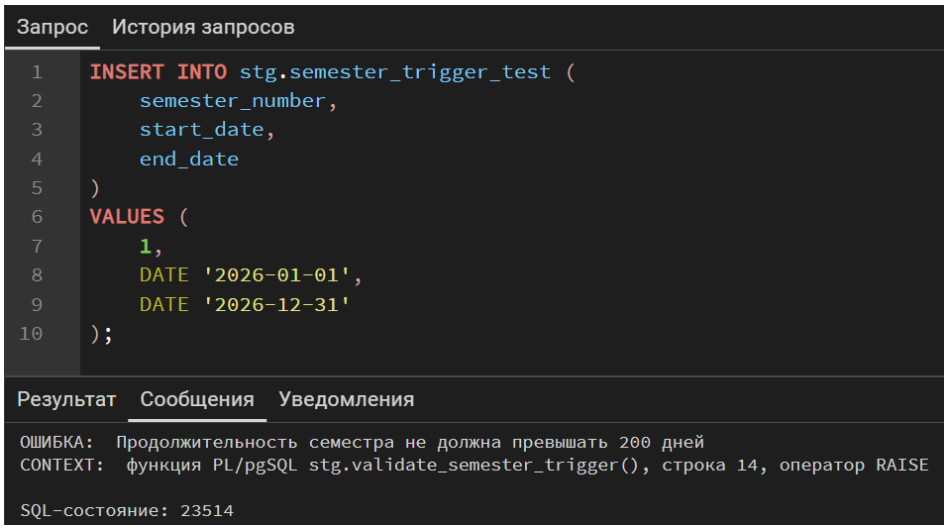


Рисунок 24 – Проверка корректности триггера вставкой невалидной продолжительности семестра

Сравнение проверок показало, что триггер снова выдает более понятное сообщение об ошибке.

Была проведена проверка производительности CHECK и триггера на вставке 10 тыс. строк (Рисунок 25–26).

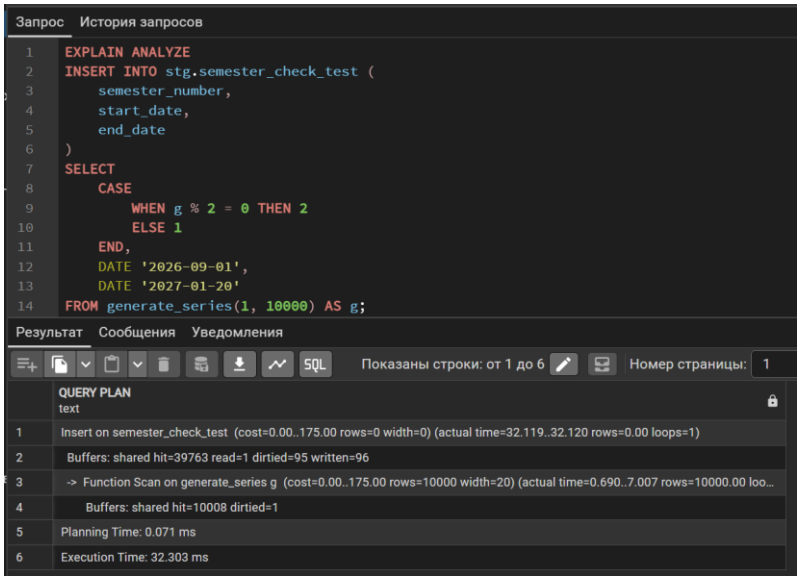


Рисунок 25 – Проверка производительности CHECK вставкой 10 тыс. строк

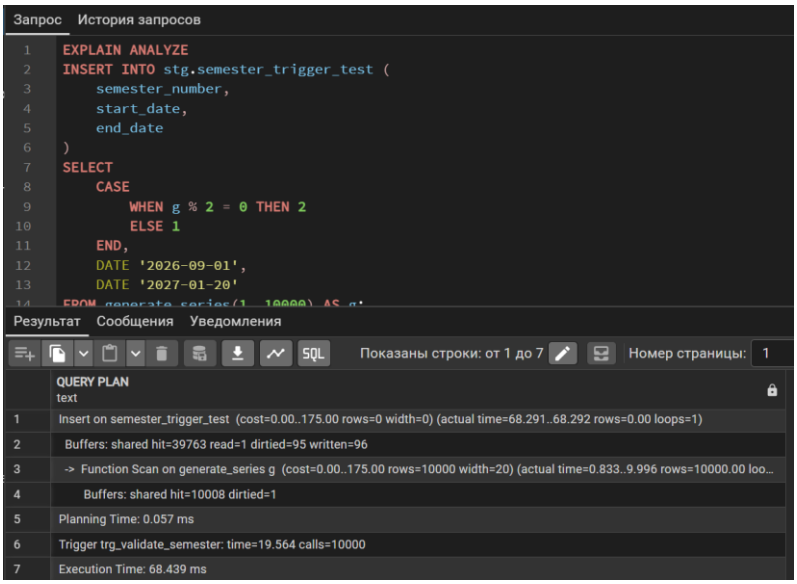


Рисунок 26 – Проверка производительности триггера вставкой 10 тыс. строк

Сравнение проверок производительности показало, что триггер оказался в 2 раза медленнее.

Исходя из всех проверок, для рассматриваемого сложного бизнес-правила подходящим вариантом был зафиксирован CHECK.

Журнал вызовов для SECURITY DEFINER-функций

Была добавлена таблица audit.function_calls (Рисунок 27).

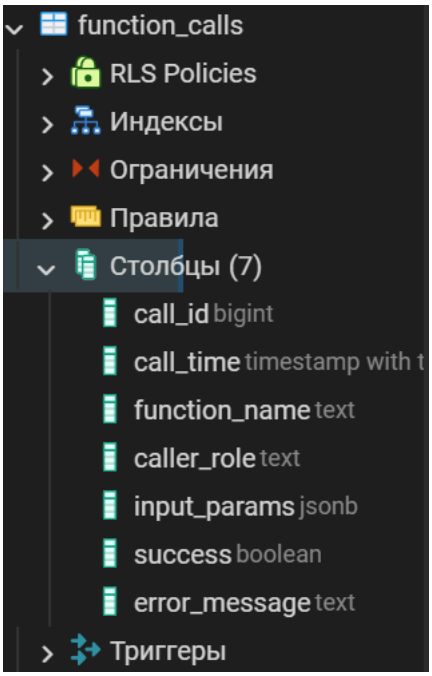


Рисунок 27 – Таблица журнала вызовов защищенных функций

Была проведена проверка журналирования вызовов защищенных функций (Рисунок 28).

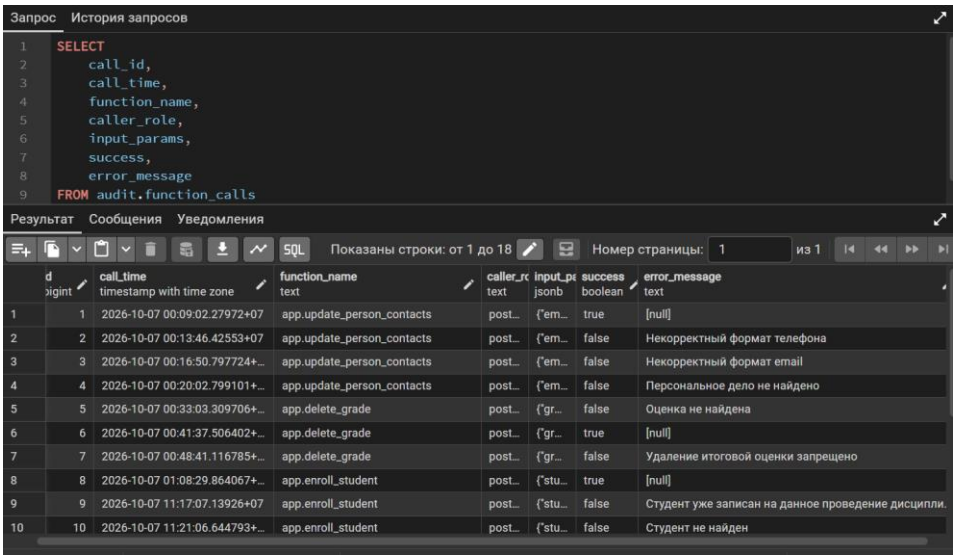


Рисунок 28 – Журнал вызовов защищенных функций

Проверка показала, что все вызовы корректно логируются.

Вывод

В ходе лабораторной работы было реализовано точечное повышение привилегий с использованием функций SECURITY DEFINER. Для чувствительных операций были созданы отдельные функции изменения контактных данных, удаления оценок и записи студентов на проведение дисциплины. Также внутри функций были добавлены проверки входных данных.

Проведено тестирование разрешённых и запрещённых действий для ролей, а также проверена обработка корректных и некорректных вызовов функций. Все обращения к защищённым функциям фиксируются в таблице audit.function_calls.

Также было выполнено сравнение ограничения CHECK и триггера на одном бизнес-правиле. Оба подхода корректно блокировали недопустимые данные, при этом триггер выдавал более понятные сообщения об ошибках, а CHECK показал лучшую производительность при вставке 10 000 строк. Для рассматриваемого правила в качестве основного варианта был выбран CHECK.

Приложение А

```
-- app.update_person_contacts - функция для изменения телефона и электронной почты

CREATE OR REPLACE FUNCTION app.update_person_contacts(p_id bigint, p_phone text,
p_email text)
RETURNS text
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = pg_catalog
AS $$
BEGIN

if p_id is null or p_id <= 0 then
    perform audit.log_function_call(
    'app.update_person_contacts',session_user,
    jsonb_build_object('person_id',p_id),false,'Неверный id');
    return 'Ошибка: неверный id';
end if;

if p_phone is not null and p_phone !~ '^[0-9+-]{5,20}$' then
    perform audit.log_function_call(
    'app.update_person_contacts',session_user,
    jsonb_build_object('person_id',p_id),false,'Неверный телефон');
    return 'Ошибка: неверный телефон';
end if;

if p_email is not null and p_email not like '%@%.%' then
    perform audit.log_function_call(
    'app.update_person_contacts',session_user,
    jsonb_build_object('person_id',p_id),false,'Неверный email');
    return 'Ошибка: неверный email';
end if;

if not exists(select 1 from app.personal_files where person_id=p_id) then
    perform audit.log_function_call(
    'app.update_person_contacts',session_user,
    jsonb_build_object('person_id',p_id),false,'Персональное дело не найдено');
    return 'Ошибка: персональное дело не найдено';
end if;

update app.personal_files
set phone=p_phone, email=p_email
where person_id=p_id;
```

```
perform audit.log_function_call(
'app.update_person_contacts',session_user,
jsonb_build_object('person_id',p_id,'phone',p_phone,'email',p_email),true,null);

return 'Данные изменены';

END;
$$;
```

-- app.delete_grade – функция, которая разрешает удаление оценки с запретом удаления итоговых оценок

```
CREATE OR REPLACE FUNCTION app.delete_grade(p_id bigint)
RETURNS text
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = pg_catalog
AS $$
DECLARE
v_type text;
BEGIN

if p_id is null or p_id <= 0 then
    perform audit.log_function_call(
    'app.delete_grade',session_user,
    jsonb_build_object('grade_id',p_id),false,'Неверный id');
    return 'Ошибка: неверный id';
end if;

select grade_type into v_type
from app.grades
where grade_id=p_id;

if not found then
    perform audit.log_function_call(
    'app.delete_grade',session_user,
    jsonb_build_object('grade_id',p_id),false,'Оценка не найдена');
    return 'Ошибка: оценка не найдена';
end if;

if v_type='final' then
    perform audit.log_function_call(
    'app.delete_grade',session_user,
    jsonb_build_object('grade_id',p_id),false,'Итоговую оценку удалить нельзя');
    return 'Ошибка: итоговую оценку удалить нельзя';
end if;
```

```
    delete from app.grades
    where grade_id=p_id;

    perform audit.log_function_call(
    'app.delete_grade',session_user,
    jsonb_build_object('grade_id',p_id),true,null);

    return 'Оценка удалена';

    END;
    $$;
```

-- app.enroll_student – функция для записи студента на проведение дисциплины

```
    CREATE OR REPLACE FUNCTION app.enroll_student(p_student_id bigint, p_course_id
bigint)
    RETURNS text
    LANGUAGE plpgsql
    SECURITY DEFINER
    SET search_path = pg_catalog
    AS $$
    DECLARE
    v_student_group bigint;
    v_course_group bigint;
    v_status text;
    BEGIN

    if p_student_id is null or p_student_id <= 0
    or p_course_id is null or p_course_id <= 0 then
        perform audit.log_function_call(
        'app.enroll_student',session_user,
        jsonb_build_object('student_id',p_student_id,'course_id',p_course_id),
        false,'Неверный id');
        return 'Ошибка: неверный id';
    end if;

    select group_id,status into v_student_group,v_status
    from app.students
    where student_id=p_student_id;

    if not found then
        perform audit.log_function_call(
        'app.enroll_student',session_user,
        jsonb_build_object('student_id',p_student_id,'course_id',p_course_id),
        false,'Студент не найден');
```

```
    return 'Ошибка: студент не найден';
end if;

if v_status<>'active' then
    perform audit.log_function_call(
    'app.enroll_student',session_user,
    jsonb_build_object('student_id',p_student_id,'course_id',p_course_id),
    false,'Студент не активен');
    return 'Ошибка: студент не активен';
end if;

select group_id into v_course_group
from app.course_offerings
where course_offering_id=p_course_id;

if not found then
    perform audit.log_function_call(
    'app.enroll_student',session_user,
    jsonb_build_object('student_id',p_student_id,'course_id',p_course_id),
    false,'Дисциплина не найдена');
    return 'Ошибка: дисциплина не найдена';
end if;

if v_student_group<>v_course_group then
    perform audit.log_function_call(
    'app.enroll_student',session_user,
    jsonb_build_object('student_id',p_student_id,'course_id',p_course_id),
    false,'Другая учебная группа');
    return 'Ошибка: другая учебная группа';
end if;

if exists(
    select 1 from app.enrollments
    where student_id=p_student_id
    and course_offering_id=p_course_id
) then
    perform audit.log_function_call(
    'app.enroll_student',session_user,
    jsonb_build_object('student_id',p_student_id,'course_id',p_course_id),
    false,'Студент уже записан');
    return 'Ошибка: студент уже записан';
end if;

insert into app.enrollments(course_offering_id,student_id,status)
values(p_course_id,p_student_id,'active');

perform audit.log_function_call(
```

```
'app.enroll_student',session_user,
jsonb_build_object('student_id',p_student_id,'course_id',p_course_id),
true,null);

return 'Студент записан';

END;
$$;
```

Приложение В

```
-- Создание тестовых таблиц для сравнения CHECK и триггера

CREATE TABLE stg.semester_check_test (
    test_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    semester_number INTEGER NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,

    CONSTRAINT chk_semester_rules
    CHECK (
        semester_number IN (1, 2)
        AND end_date > start_date
        AND end_date - start_date <= 200
    )
);

CREATE TABLE stg.semester_trigger_test (
    test_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    semester_number INTEGER NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL
);


-- Функция проверки бизнес-правил учебного семестра

CREATE OR REPLACE FUNCTION stg.validate_semester_trigger()
RETURNS trigger
LANGUAGE plpgsql
AS $$
BEGIN

if NEW.semester_number <> 1 and NEW.semester_number <> 2 then
    raise exception 'Номер семестра должен быть 1 или 2'
    using errcode='23514';
end if;

if NEW.end_date <= NEW.start_date then
    raise exception 'Дата окончания должна быть позже даты начала'
    using errcode='23514';
end if;

if NEW.end_date - NEW.start_date > 200 then
```

```
    raise exception 'Семестр не может длиться больше 200 дней'
    using errcode='23514';
end if;

return NEW;

END;
$$;
```

-- Триггер проверки данных при добавлении и изменении учебного семестра

```
CREATE TRIGGER trg_validate_semester
BEFORE INSERT OR UPDATE
ON stg.semester_trigger_test
FOR EACH ROW
EXECUTE FUNCTION stg.validate_semester_trigger();
```